

Reviewer Pack — Additive Energy Inverse Proportional to Disjointness Discrep...

Entry 4ab417bc1c0b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 22:42:01 UTC

Additive Energy Inverse Proportional to Disjointness Discrepancy

Entry ID: 4ab417bc1c0b Verdict: INCONCLUSIVE Recorded: 2026-05-10 22:42:01 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): Communication Complexity of Disjointness

Statement:

For a disjointness function $f: \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$, the additive energy $E(f)$ of its communication matrix satisfies $E(f) = \Theta(1/\text{disc}(f))$, where $\text{disc}(f)$ is the discrepancy of f under uniform distributions.

Rationale (proposer’s reasoning):

Additive energy captures the number of additive configurations in the communication matrix, which directly influences the discrepancy via the structure of overlapping supports. High additive energy implies structured overlaps, reducing discrepancy.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cfa278f8b763698c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_disjointness_instance(n):
        A = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
        B = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
        return A, B

    def communication_matrix(A, B):
        n = len(A)
        C = [[A[i][j] & B[j][i] for j in range(n)] for i in range(n)]
        return C

    def additive_energy(C):
        n = len(C)
        count = 0
        for i in range(n):
            for j in range(i, n):
                for k in range(j, n):
                    for l in range(k, n):
                        if C[i][j] + C[j][k] + C[k][l] + C[l][i] >= 3:
                            count += 1
        return count

    def discrepancy(C):
        n = len(C)
        max_cut = 0
        for i in range(n):
            for j in range(i, n):
                cut = sum(C[i][k] & C[j][k] for k in range(n))
                if cut > max_cut:
                    max_cut = cut
        return max_cut

    A, B = generate_disjointness_instance(5)
    C = communication_matrix(A, B)
```

```

E = additive_energy(C)
disc = discrepancy(C)

metric_name = "additive_energy_over_disc"
metric_value = E / disc if disc != 0 else float('inf')
instances_tested = 1
conjecture_holds = abs(metric_value - 4) < 1e-6 # Assuming constant is close to 4 for n=5
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in input().split()] if len(sys.argv) > 1 else list(range(2, 30)) # Default to
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44b6282a.py", line 73, in <module>

```

    seeds = [int(x) for x in input().split()] if len(sys.argv) > 1 else list(range(2, 30)) # Default to
                ^^^^^^^

```

EOFError: EOF when reading a line

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of support fraction or counterexamples | next: Fix the test to handle input properly and re-run with sufficient seed coverage

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111105
2	propose	ollama_remote	qwen3:8b	0	0	80603
3	preregistration	ollama_remote	qwen3:8b	0	0	24062
4	novelty	ollama_remote	qwen3:8b	0	0	21576
5	novelty	ollama_remote	qwen3:8b	0	0	14260
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13006
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8603
8	judge	ollama_remote	qwen3:8b	0	0	15211

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 288427 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/4ab417bc1c0b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4ab417bc1c0b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4ab417bc1c0b.tar.gz (if generated)